MiniJava Compiler – Sprint 2 Guide: AST and Parser

Author: Deepanshu Chaurasia (Deepu) | **Sprint:** 2 of 4 | **Planned dates:** 4 to 10 Nov 2026 (start earlier if Sprint 1 is done) | **Covers:** FR-11 to FR-20 and FR-41

> **Before you start:** Sprint 1 must be finished: `mvn test` is green (19 tests), the code is merged to `main`, and tag `sprint-1` exists. Sprint 2 uses `Token`, `TokenType`, `Lexer` and `CompileException` exactly as they are. Do **not** rename any file, package or method in this guide. Type the code yourself and read the explanation of each part.

1. Sprint 2 Goal

What: The parser builds an **AST** (Abstract Syntax Tree) from the tokens, and it reports clear syntax errors. **Why:** the lexer only gives a flat list of tokens. A flat list does not say that `3 * 4` must be calculated before `+ 2`. The tree does.

`Source text` → Lexer → `List<Token>` → **Parser** → `AST` (you are here)

Sprint 2 is done when all of these are true:

- [ ] `mvn test` shows **50 tests, 0 failures** (19 from Sprint 1 and 31 new)
- [ ] `--ast examples/hello.mj` prints the expected tree lines
- [ ] The two syntax error examples give the expected messages with line and column
- [ ] Everything is committed, merged to `main`, and tagged `sprint-2`

Fixed rules from the design (already decided, so nothing changes later):

- The parser is hand-written recursive descent: one method per grammar rule.
- It stops at the first error.
- "Expected ..." errors point just *after the previous token*. This is why a missing `;` is reported on the line where it is missing.
- Operators are enums. Every AST node keeps `line` and `column` of the most helpful token.
- `x = 5;` (assignment) is part of V1.

2. Files in this Sprint (where each file goes)

All paths start with `src/main/java/com/deepu/minijava/` unless written otherwise.

#	File	Step
1	`diagnostics/ParseException.java`	1
2	`parser/ast/BinaryOperator.java`	2
3	`parser/ast/UnaryOperator.java`	2
4	`parser/ast/Expr.java`	2
5	`parser/ast/Stmt.java`	2
6	`parser/ast/NumberExpr.java`	2
7	`parser/ast/VariableExpr.java`	2
8	`parser/ast/BinaryExpr.java`	2
9	`parser/ast/UnaryExpr.java`	2
10	`parser/ast/VariableDeclaration.java`	2
11	`parser/ast/AssignStatement.java`	2
12	`parser/ast/PrintStatement.java`	2
13	`parser/ast/ExpressionStatement.java`	2
14	`parser/ast/Program.java`	2
15	`parser/ast/AstPrinter.java`	3
16	`parser/Parser.java`	4
17	`src/test/java/com/deepu/minijava/parser/ParserTest.java`	5
18	`Main.java` (replace the whole file)	6
19	`examples/variables.mj`	6
20	`examples/errors/missing_semicolon.mj`	6
21	`examples/errors/unmatched_paren.mj`	6

3. New Java Features in this Sprint (quick meanings)

Feature	Simple meaning	Where
`sealed interface ... permits`	An interface that only the listed classes may implement. The compiler then knows *all* possible kinds	`Expr`, `Stmt`
`record` with a compact constructor	A data class where you can add a small check or copy step in the constructor	`Program`
`switch` with type patterns	A `switch` that checks which *kind* of object you have. On a sealed type, the compiler warns if you forget one kind	`AstPrinter`
`TokenType...` (varargs)	A method that accepts any number of values of that type	`Parser.match`
`import static`	Lets you write `PLUS` instead of `TokenType.PLUS`	`Parser`

4. Idea First: How the Parser Works (read this before coding)

What: The parser reads the token list from left to right and builds a tree. **Analogy:** a sentence diagram from school (subject, verb, object), or a family tree. Each part of the program becomes one *node*, and nodes sit inside bigger nodes.

Example: `int x = 2 + 3 * 4;` becomes

```
VariableDeclaration(x)
  └─ BinaryExpr(+)
       ├─ NumberExpr(2)
       └─ BinaryExpr(*)
            ├─ NumberExpr(3)
            └─ NumberExpr(4)
```

`3 * 4` sits deeper, so it is calculated first. The tree *stores* the precedence.

4.1 Recursive descent: one method for each grammar rule

The grammar (from the SRS) says what a correct program looks like. For each rule we write one method with the same name. Rules use other rules, so the methods call each other.

Grammar rule	Method	What it reads
`program`	`parse()`	Many statements until the end
`statement`	`statement()`	Looks at the first token and picks one of the four below
`varDecl`	`varDeclaration()`	`int x = expr ;`
`assignStmt`	`assignmentStatement()`	`x = expr ;`
`printStmt`	`printStatement()`	`print ( expr ) ;`
`exprStmt`	`expressionStatement()`	`expr ;`
`expression`	`expression()`	`+` and `-`
`term`	`term()`	`*` and `/`
`unary`	`unary()`	A minus sign in front, like `-5`
`primary`	`primary()`	A number, a variable name, or `( expr )`

Why precedence works: `expression()` calls `term()`, and `term()` calls `unary()`, and `unary()` calls `primary()`. The deeper a rule is, the tighter it binds. So `*` is always built deeper in the tree than `+`.

Trace for `2 + 3 * 4`:

Step	Method running	What happens
1	`expression()`	Calls `term()` to get its left side
2	`term()` then `unary()` then `primary()`	Reads `2`. `term()` sees `+`, which is not `*` or `/`, so it returns `2`
3	`expression()`	Sees `+`. Takes it. Calls `term()` for the right side

Step	Method running	What happens
4	`term()`	`unary()` and `primary()` read `3`. Then `term()` sees `*`, takes it, reads `4`
5	`term()`	Builds `BinaryExpr(3 * 4)` and returns it
6	`expression()`	Builds `BinaryExpr(2 + (3 * 4))`

Why left-to-right works: `expression()` has a `while` loop. For `10 - 4 - 3`: left is `10`. It sees `-` and reads `4`, so left becomes `(10 - 4)`. It sees `-` again and reads `3`, so left becomes `((10 - 4) - 3)`. The answer is 3, which is correct. Without the loop we would get `10 - (4 - 3)` = 9, which is wrong.

Unary minus: `unary()` calls *itself* after the `-`. So `- -5` works too.

Assignment or expression statement? Both can start with a name: `x = 5;` and `x + 1;`. The parser looks one token further. If the first token is a name and the next token is `=`, it is an assignment. This is called *lookahead*, and it is the helper `checkNext(...)`.

Errors: `consume(type, message)` means "the next token *must* be this type". If not, it throws a `ParseException`. By the fixed rule, the error position is just after the previous token.

Step 0: Create a branch

```
git checkout main
git pull
git checkout -b feature/sprint-2-parser
```

Create the packages `com.deepu.minijava.parser` and `com.deepu.minijava.parser.ast` under `src/main/java`, and `com.deepu.minijava.parser` under `src/test/java`.

Step 1: ParseException

What: The error type for syntax errors. **Why:** same style as `LexerException`, so `Main` handles it with the same `catch`.

Create `diagnostics/ParseException.java`:

```
package com.deepu.minijava.diagnostics;

/** Thrown by the Parser when the tokens do not follow the grammar of
MiniJava. */
public class ParseException extends CompileException {

    public ParseException(String message, int line, int column) {
        super("Syntax", message, line, column);
    }
}
```

Check: no red lines. Errors will look like `[Syntax Error] line 1, column 10: ...`.

Step 2: AST classes (parser/ast package)

What: One small class for each kind of node. **Why:** the parser builds them, and in Sprint 3 the semantic analyzer and code generator read them. **Important:** this package imports nothing from the lexer, so the AST stays independent (requirement FR-20).

The position rule: `line` and `column` point to the most helpful token for error messages. That is the *name* for declarations and assignments, the *operator* for binary and unary expressions, the `print` keyword for print statements, and the first token for the rest.

Create `parser/ast/BinaryOperator.java`:

```
package com.deepu.minijava.parser.ast;

/** The operators that take two values. Comparison operators are added in V2.
*/
public enum BinaryOperator {
    ADD("+"), SUBTRACT("-"), MULTIPLY("*"), DIVIDE("/");

    private final String symbol;

    BinaryOperator(String symbol) {
        this.symbol = symbol;
    }

    public String symbol() {
        return symbol;
    }
}
```

Create `parser/ast/UnaryOperator.java`:

```
package com.deepu.minijava.parser.ast;

/** The operators that take one value. NOT is added in V2. */
public enum UnaryOperator {
    NEGATE
}
```

Create `parser/ast/Expr.java`:

```
package com.deepu.minijava.parser.ast;

/** An expression: something that has a value, like 2 + 3. */
public sealed interface Expr permits NumberExpr, VariableExpr, BinaryExpr,
UnaryExpr {
}
```

Create `parser/ast/Stmt.java`:

```
package com.deepu.minijava.parser.ast;

/** A statement: one complete instruction, like int x = 5; */
public sealed interface Stmt permits VariableDeclaration, AssignStatement,
PrintStatement, ExpressionStatement {
}
```

Create `parser/ast/NumberExpr.java`:

```
package com.deepu.minijava.parser.ast;

/** A whole number, like 21. */
public record NumberExpr(int value, int line, int column) implements Expr {
}
```

Create `parser/ast/VariableExpr.java`:

```
package com.deepu.minijava.parser.ast;

/** Using a variable, like age in print(age). */
public record VariableExpr(String name, int line, int column) implements Expr
{
}
```

Create `parser/ast/BinaryExpr.java`:

```
package com.deepu.minijava.parser.ast;

/** Two values with an operator between them. Line and column are the
position of the operator. */
public record BinaryExpr(Expr left, BinaryOperator operator, Expr right, int
line, int column) implements Expr {
}
```

Create `parser/ast/UnaryExpr.java`:

```
package com.deepu.minijava.parser.ast;

/** One value with an operator in front, like -5. Line and column are the
position of the operator. */
public record UnaryExpr(UnaryOperator operator, Expr operand, int line, int
column) implements Expr {
}
```

Create `parser/ast/VariableDeclaration.java`:

```
package com.deepu.minijava.parser.ast;

/** int name = initializer; Line and column are the position of the variable
name. */
public record VariableDeclaration(String name, Expr initializer, int line,
int column) implements Stmt {
}
```

Create `parser/ast/AssignStatement.java`:

```
package com.deepu.minijava.parser.ast;

/** name = value; Line and column are the position of the variable name. */
public record AssignStatement(String name, Expr value, int line, int column)
implements Stmt {
}
```

Create `parser/ast/PrintStatement.java`:

```
package com.deepu.minijava.parser.ast;

/** print(expression); Line and column are the position of the print keyword.
*/
public record PrintStatement(Expr expression, int line, int column)
implements Stmt {
}
```

Create `parser/ast/ExpressionStatement.java`:

```
package com.deepu.minijava.parser.ast;

/** An expression used as a statement, like 5 + 3; Line and column are the
position of its first token. */
public record ExpressionStatement(Expr expression, int line, int column)
implements Stmt {
}
```

Create `parser/ast/Program.java`:

```
package com.deepu.minijava.parser.ast;

import java.util.List;

/** The whole program: a list of statements. */
public record Program(List<Stmt> statements) {

    public Program {
        statements = List.copyOf(statements);   // keep our own copy, so
nobody can change it later
    }
}
```

Important parts:

- A **sealed interface** with `permits` is a closed list of kinds. An `Expr` can only be a `NumberExpr`, `VariableExpr`, `BinaryExpr` or `UnaryExpr`. This is what lets the compiler check our `switch` later.
- A `BinaryExpr` holds two `Expr` objects, and they can be `BinaryExpr` objects again. That is how a tree is built with just a few classes.
- There is no node for brackets. The tree shape already shows what was grouped, so `(2 + 3) * 4` is just a `BinaryExpr` whose left side is a `BinaryExpr`.
- `List.copyOf` in `Program` makes the list read-only. A tree should not change after it is built.

Check: no red lines. If IntelliJ says a record is "not allowed to extend sealed class", check that its name is in the `permits` list and that it is in the same package.

Step 3: AstPrinter

What: A small helper that turns the tree into short text, one line per statement. **Why:** we need to *see* the tree to test the parser, and it will help you learn. It also makes the tests short: instead of building big trees by hand, we compare text like `(+ 2 (* 3 4))`.

The text format:

Node	Printed as	
number	`21`	
variable	`age`	
binary	`(+ left right)`	
unary minus	`(neg value)`	
declaration	`(var x value)`	
assignment	`(assign x value)`	
print	`(print value)`	
expression statement	`(expr value)`	

Create `parser/ast/AstPrinter.java`:

```
package com.deepu.minijava.parser.ast;

import java.util.stream.Collectors;

/** Turns an AST into short readable text. Used by the --ast option and by
the tests. */
public final class AstPrinter {

    private AstPrinter() {
    }

    /** One line for each statement. */
    public static String print(Program program) {
        return program.statements().stream()
                .map(statement -> printStatement(statement))
                .collect(Collectors.joining("\n"));
    }
```

```
    private static String printStatement(Stmt statement) {
        return switch (statement) {
            case VariableDeclaration d -> "(var " + d.name() + " " +
printExpression(d.initializer()) + ")";
            case AssignStatement a -> "(assign " + a.name() + " " +
printExpression(a.value()) + ")";
            case PrintStatement p -> "(print " +
printExpression(p.expression()) + ")";
            case ExpressionStatement e -> "(expr " +
printExpression(e.expression()) + ")";
        };
    }

    private static String printExpression(Expr expression) {
        return switch (expression) {
            case NumberExpr n -> String.valueOf(n.value());
            case VariableExpr v -> v.name();
            case BinaryExpr b -> "(" + b.operator().symbol() + " "
                    + printExpression(b.left()) + " " +
printExpression(b.right()) + ")";
            case UnaryExpr u -> "(" + operatorName(u.operator()) + " " +
printExpression(u.operand()) + ")";
        };
    }

    private static String operatorName(UnaryOperator operator) {
        return switch (operator) {
            case NEGATE -> "neg";
        };
    }
}
```

Important parts:

- `case VariableDeclaration d -> ...` means: "if the statement is a `VariableDeclaration`, call it `d` and do this". There is **no `default`**. Because `Stmt` is sealed, Java knows the four cases cover everything. If you add a new statement kind later and forget a case, the code will not compile. This is the safety net we chose in design decision D-3.

- The printer calls itself for the inside parts. Trees are *recursive*, so the code that walks them is recursive too. You will see the same shape in the semantic analyzer and code generator in Sprint 3.

Check: no red lines.

Step 4: The Parser

What: The class that reads tokens and builds the AST. **Why:** it is stage 2 of the compiler. **How:** the grammar from Section 4, one method per rule, and a few small helper methods. Read the table below the code, it explains every helper.

Create `parser/Parser.java`:

```
package com.deepu.minijava.parser;

import com.deepu.minijava.diagnostics.ParseException;
import com.deepu.minijava.lexer.Token;
import com.deepu.minijava.lexer.TokenType;
import com.deepu.minijava.parser.ast.*;

import java.util.ArrayList;
import java.util.List;

import static com.deepu.minijava.lexer.TokenType.*;

/**
 * Stage 2 of the compiler: turns a list of tokens into an AST.
 * It is a hand-written recursive-descent parser: one method for each grammar
rule.
 * The token list must end with an EOF token (the Lexer always adds it).
 * The parser stops at the first error.
 */
public class Parser {

    private final List<Token> tokens;
    private int current = 0;   // index of the token we are looking at

    public Parser(List<Token> tokens) {
        this.tokens = tokens;
    }
```

```
    /** program = { statement } EOF ; */
    public Program parse() {
        List<Stmt> statements = new ArrayList<>();
        while (!isAtEnd()) {
            statements.add(statement());
        }
        return new Program(statements);
    }

    // ---------- Statements ----------

    /** statement = varDecl | assignStmt | printStmt | exprStmt ; */
    private Stmt statement() {
        if (check(INT)) {
            return varDeclaration();
        }
        if (check(PRINT)) {
            return printStatement();
        }
        if (check(IDENTIFIER) && checkNext(EQUAL)) {
            return assignmentStatement();
        }
        // These words and symbols are reserved for V2 (control flow and
blocks).
        if (check(IF) || check(ELSE) || check(WHILE) || check(LEFT_BRACE) ||
check(RIGHT_BRACE)) {
            throw errorAtCurrent("Expected a statement");
        }
        return expressionStatement();
    }

    /** varDecl = "int" IDENTIFIER "=" expression ";" ; */
    private Stmt varDeclaration() {
        advance();   // the word int
        Token name = consume(IDENTIFIER, "Expected a variable name after
'int'");
        consume(EQUAL, "Expected '=' after the variable name");
        Expr initializer = expression();
        consume(SEMICOLON, "Expected ';' after the variable declaration");
        return new VariableDeclaration(name.lexeme(), initializer,
name.line(), name.column());
    }
```

```
    /** assignStmt = IDENTIFIER "=" expression ";" ; */
    private Stmt assignmentStatement() {
        Token name = advance();   // the variable name
        advance();                // the = sign
        Expr value = expression();
        consume(SEMICOLON, "Expected ';' after the assignment");
        return new AssignStatement(name.lexeme(), value, name.line(),
name.column());
    }

    /** printStmt = "print" "(" expression ")" ";" ; */
    private Stmt printStatement() {
        Token keyword = advance();   // the word print
        consume(LEFT_PAREN, "Expected '(' after 'print'");
        Expr value = expression();
        consume(RIGHT_PAREN, "Expected ')' after the value to print");
        consume(SEMICOLON, "Expected ';' after the print statement");
        return new PrintStatement(value, keyword.line(), keyword.column());
    }

    /** exprStmt = expression ";" ; */
    private Stmt expressionStatement() {
        Token first = peek();
        Expr expression = expression();
        consume(SEMICOLON, "Expected ';' after the expression");
        return new ExpressionStatement(expression, first.line(),
first.column());
    }

    // ---------- Expressions (from the weakest to the strongest operator) --
--------

    /** expression = term { ( "+" | "-" ) term } ; */
    private Expr expression() {
        Expr left = term();
        while (match(PLUS, MINUS)) {
            Token operator = previous();
            Expr right = term();
            BinaryOperator kind = operator.type() == PLUS ?
BinaryOperator.ADD : BinaryOperator.SUBTRACT;
            left = new BinaryExpr(left, kind, right, operator.line(),
operator.column());
        }
```

```
        return left;
    }

    /** term = unary { ( "*" | "/" ) unary } ; */
    private Expr term() {
        Expr left = unary();
        while (match(STAR, SLASH)) {
            Token operator = previous();
            Expr right = unary();
            BinaryOperator kind = operator.type() == STAR ?
BinaryOperator.MULTIPLY : BinaryOperator.DIVIDE;
            left = new BinaryExpr(left, kind, right, operator.line(),
operator.column());
        }
        return left;
    }

    /** unary = "-" unary | primary ; */
    private Expr unary() {
        if (match(MINUS)) {
            Token operator = previous();
            Expr operand = unary();   // calls itself, so - -5 works
            return new UnaryExpr(UnaryOperator.NEGATE, operand,
operator.line(), operator.column());
        }
        return primary();
    }

    /** primary = INTEGER | IDENTIFIER | "(" expression ")" ; */
    private Expr primary() {
        if (match(INTEGER)) {
            Token number = previous();
            // The Lexer already checked that the number fits in an int.
            return new NumberExpr(Integer.parseInt(number.lexeme()),
number.line(), number.column());
        }
        if (match(IDENTIFIER)) {
            Token name = previous();
            return new VariableExpr(name.lexeme(), name.line(),
name.column());
        }
        if (match(LEFT_PAREN)) {
            Expr inside = expression();
```

```
            consume(RIGHT_PAREN, "Expected ')' after the expression");
            return inside;   // brackets need no node of their own: the tree
shape shows the grouping
        }
        throw errorAtCurrent("Expected an expression");
    }

    // ---------- Helper methods ----------

    /** If the current token has one of these types, move forward and return
true. */
    private boolean match(TokenType... types) {
        for (TokenType type : types) {
            if (check(type)) {
                advance();
                return true;
            }
        }
        return false;
    }

    /** The current token MUST have this type. If not, throw a syntax error.
*/
    private Token consume(TokenType type, String expected) {
        if (check(type)) {
            return advance();
        }
        throw errorAfterPrevious(expected);
    }

    /** Is the current token of this type? Does not move. */
    private boolean check(TokenType type) {
        return peek().type() == type;
    }

    /** Is the token after the current one of this type? Does not move. (One
token of lookahead.) */
    private boolean checkNext(TokenType type) {
        if (isAtEnd()) {
            return false;
        }
        return tokens.get(current + 1).type() == type;
    }
```

```
    /** Moves forward by one token and returns the token we just left. */
    private Token advance() {
        if (!isAtEnd()) {
            current++;
        }
        return previous();
    }

    private Token peek() {
        return tokens.get(current);
    }

    private Token previous() {
        return tokens.get(current - 1);
    }

    private boolean isAtEnd() {
        return peek().type() == EOF;
    }

    // ---------- Errors ----------

    /** The error points at the token we are looking at. */
    private ParseException errorAtCurrent(String expected) {
        Token found = peek();
        return new ParseException(expected + ", but found " +
describe(found), found.line(), found.column());
    }

    /** The error points just after the previous token. A missing ; is then
shown on the line where it is missing. */
    private ParseException errorAfterPrevious(String expected) {
        if (current == 0) {
            return errorAtCurrent(expected);
        }
        Token previous = previous();
        return new ParseException(expected + ", but found " +
describe(peek()), previous.line(), previous.endColumn());
    }

    private static String describe(Token token) {
```

```
        return token.type() == EOF ? "end of file" : "'" + token.lexeme() +
"'";
    }
}
```

The helper methods (important):

Helper	What it does	Analogy
`peek()`	Looks at the current token without moving	Reading the next word of a sentence
`advance()`	Moves forward by one token	Turning to the next word
`check(type)`	"Is the current token of this type?"	A yes or no question
`checkNext(type)`	Same, for the token *after* the current one	Looking two words ahead
`match(types...)`	If the current token has one of these types, take it and say yes	"Is it a plus or a minus? If yes, take it"
`consume(type, text)`	The token MUST be this type, or throw an error	A rule that must be followed
`errorAfterPrevious`	Builds the error and points just after the previous token	Pointing to the exact place where a word is missing

Important parts of the design:

- **`match` is used for optional things** (`+`, `-`, `*`, `/`). **`consume` is used for required things** (`;`, `)`, `=`).
- **The `while` loops** in `expression()` and `term()` are what make `10 - 4 - 3` come out as `(10 - 4) - 3`.
- **`unary()` calls itself.** This is the recursion that handles `- -5`.
- **Every statement method eats at least one token or throws an error.** So the loop in `parse()` can never run forever.
- **Errors:** `consume` uses the position *after the previous token*. `primary()` and reserved words use the *current token*, because there the current token itself is the problem.
- **`if`, `else`, `while`, `{` and `}`** give "Expected a statement" in V1. They are reserved for V2.

Check: no red lines. If IntelliJ says an import is unused or ambiguous, check that you wrote `import static com.deepu.minijava.lexer.TokenType.*;` exactly.

Step 5: Tests

What: 31 tests for the parser. **Why:** they prove precedence, left-to-right order, positions and every error message. **How:** the helper `ast(...)` parses text and prints the tree with `AstPrinter`. The helper `assertParseError(...)` checks the error message, line and column.

Create `src/test/java/com/deepu/minijava/parser/ParserTest.java`:

```
package com.deepu.minijava.parser;

import com.deepu.minijava.diagnostics.ParseException;
import com.deepu.minijava.lexer.Lexer;
import com.deepu.minijava.parser.ast.*;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class ParserTest {

    private Program parse(String source) {
        return new Parser(new Lexer(source).scanTokens()).parse();
    }

    /** Parses the source and returns the tree as short text, one line per
statement. */
    private String ast(String source) {
        return AstPrinter.print(parse(source));
    }

    private void assertParseError(String source, String message, int line,
int column) {
        ParseException e = assertThrows(ParseException.class, () ->
parse(source));
        assertEquals(message, e.getMessage());
        assertEquals(line, e.getLine());
        assertEquals(column, e.getColumn());
    }
```

```
    // ---------- Valid programs ----------

    @Test
    void emptyProgram() {
        Program program = parse("");
        assertTrue(program.statements().isEmpty());
        assertEquals("", AstPrinter.print(program));
    }

    @Test
    void variableDeclaration() {
        assertEquals("(var age 21)", ast("int age = 21;"));
    }

    @Test
    void printStatement() {
        assertEquals("(print age)", ast("print(age);"));
    }

    @Test
    void assignment() {
        assertEquals("(assign x 10)", ast("x = 10;"));
    }

    @Test
    void expressionStatement() {
        assertEquals("(expr (+ 5 3))", ast("5 + 3;"));
    }

    @Test
    void multiplicationBeforeAddition() {
        assertEquals("(print (+ 2 (* 3 4)))", ast("print(2 + 3 * 4);"));
    }

    @Test
    void parenthesesChangePrecedence() {
        assertEquals("(print (* (+ 2 3) 4))", ast("print((2 + 3) * 4);"));
    }

    @Test
    void subtractionIsLeftToRight() {
        assertEquals("(print (- (- 10 4) 3))", ast("print(10 - 4 - 3);"));
    }
```

```
    @Test
    void divisionIsLeftToRight() {
        assertEquals("(print (/ (/ 100 10) 2))", ast("print(100 / 10 /
2);"));
    }

    @Test
    void unaryMinus() {
        assertEquals("(print (+ (neg 5) 8))", ast("print(-5 + 8);"));
    }

    @Test
    void doubleUnaryMinus() {
        assertEquals("(print (neg (neg 5)))", ast("print(- -5);"));
    }

    @Test
    void unaryBindsTighterThanMultiplication() {
        assertEquals("(print (* (neg 2) 3))", ast("print(-2 * 3);"));
    }

    @Test
    void variablesInExpression() {
        assertEquals("(var z (+ (* a b) c))", ast("int z = a * b + c;"));
    }

    @Test
    void multipleStatements() {
        String source = "int age = 21;\nprint(age);\nprint(10 + 20 * 3);";
        assertEquals("(var age 21)\n(print age)\n(print (+ 10 (* 20 3)))",
ast(source));
    }

    @Test
    void nodePositions() {
        Program program = parse("int x = 1 + 2;");
        VariableDeclaration declaration = (VariableDeclaration)
program.statements().get(0);
        assertEquals(1, declaration.line());
        assertEquals(5, declaration.column());          // position of the
name x
        BinaryExpr sum = (BinaryExpr) declaration.initializer();
```

```
        assertEquals(11, sum.column());                 // position of the +
operator
    }

    @Test
    void printStatementPosition() {
        Program program = parse("\nprint(1);");
        PrintStatement statement = (PrintStatement)
program.statements().get(0);
        assertEquals(2, statement.line());
        assertEquals(1, statement.column());
    }

    // ---------- Syntax errors ----------

    @Test
    void missingSemicolonAfterDeclaration() {
        assertParseError("int x = 5",
                "Expected ';' after the variable declaration, but found end
of file", 1, 10);
    }

    @Test
    void missingSemicolonIsReportedOnTheLineWhereItIsMissing() {
        assertParseError("int x = 5\nprint(x);",
                "Expected ';' after the variable declaration, but found
'print'", 1, 10);
    }

    @Test
    void unmatchedLeftParenthesis() {
        assertParseError("print((2 + 3);",
                "Expected ')' after the value to print, but found ';'", 1,
14);
    }

    @Test
    void unmatchedRightParenthesis() {
        assertParseError("print(2 + 3));",
                "Expected ';' after the print statement, but found ')'", 1,
13);
    }
```

```
    @Test
    void missingClosingParenthesisInGroup() {
        assertParseError("int x = (1 + 2;",
                "Expected ')' after the expression, but found ';'", 1, 15);
    }

    @Test
    void missingExpressionAfterEquals() {
        assertParseError("int x = ;", "Expected an expression, but found
';'", 1, 9);
    }

    @Test
    void incompleteExpression() {
        assertParseError("print(2 + );", "Expected an expression, but found
')'", 1, 11);
    }

    @Test
    void missingVariableName() {
        assertParseError("int = 5;", "Expected a variable name after 'int',
but found '='", 1, 4);
    }

    @Test
    void keywordCannotBeVariableName() {
        assertParseError("int print = 5;",
                "Expected a variable name after 'int', but found 'print'", 1,
4);
    }

    @Test
    void missingEqualsInDeclaration() {
        assertParseError("int x;", "Expected '=' after the variable name, but
found ';'", 1, 6);
    }

    @Test
    void missingParenAfterPrint() {
        assertParseError("print 5;", "Expected '(' after 'print', but found
'5'", 1, 6);
    }
```

```
    @Test
    void missingSemicolonAfterAssignment() {
        assertParseError("x = 5",
                "Expected ';' after the assignment, but found end of file",
1, 6);
    }

    @Test
    void ifIsNotSupportedYet() {
        assertParseError("if (1) print(1);", "Expected a statement, but found
'if'", 1, 1);
    }

    @Test
    void braceIsNotSupportedYet() {
        assertParseError("{", "Expected a statement, but found '{'", 1, 1);
    }

    @Test
    void strayRightParenthesis() {
        assertParseError(")", "Expected an expression, but found ')'", 1, 1);
    }
}
```

Important parts:

- The strings like `(+ 2 (* 3 4))` read as: "add 2 and the result of multiplying 3 and 4". If precedence were wrong, the test would show `(* (+ 2 3) 4)` instead.
- Look at `subtractionIsLeftToRight`. The answer `(- (- 10 4) 3)` proves the `while` loop works.
- The error tests check **three things**: message, line and column. Read `missingSemicolonIsReportedOnTheLineWhereItIsMissing`. The next line starts with `print`, but the error is still reported on **line 1, column 10**. That is the "just after the previous token" rule.
- You can already check the tests with the SRS list: operator precedence, missing semicolons, unmatched parentheses.

Check: no red lines. Run the tests in Step 7.

Step 6: Update Main and add examples

What: Connect the parser to `Main`, and add a new `--ast` option. **Why:** so you can run the parser on any `.mj` file and see the tree. **How:** replace the **whole** `Main.java` with the version below. Compared with Sprint 1, there are only these changes: three new imports, the `--ast` option, two parser lines, the changed usage text, and the old temporary "Lexing finished" message is replaced by a "Parsing finished" message.

Replace `src/main/java/com/deepu/minijava/Main.java`:

```
package com.deepu.minijava;

import com.deepu.minijava.diagnostics.CompileException;
import com.deepu.minijava.lexer.Lexer;
import com.deepu.minijava.lexer.Token;
import com.deepu.minijava.parser.Parser;
import com.deepu.minijava.parser.ast.AstPrinter;
import com.deepu.minijava.parser.ast.Program;

import java.io.IOException;
import java.io.PrintStream;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;

/** Entry point: Source File -> Lexer -> Parser -> (more stages are added in
later sprints) */
public class Main {

    private static final int EXIT_OK = 0;
    private static final int EXIT_ERROR = 1;   // compile error or runtime
error in the .mj program
    private static final int EXIT_USAGE = 2;   // wrong command line or
unreadable file

    public static void main(String[] args) {
        System.exit(run(args, System.out, System.err));
    }

    /** Does all the work and returns the exit code. Kept separate from
main() so tests can call it. */
    static int run(String[] args, PrintStream out, PrintStream err) {
```

```
        boolean showTokens = false;
        boolean showAst = false;
        String fileName = null;

        for (String arg : args) {
            if (arg.equals("--tokens")) {
                showTokens = true;
            } else if (arg.equals("--ast")) {
                showAst = true;
            } else if (arg.startsWith("--")) {
                err.println("Unknown option: " + arg);
                return usage(err);
            } else if (fileName == null) {
                fileName = arg;
            } else {
                err.println("Only one source file is allowed.");
                return usage(err);
            }
        }

        if (fileName == null) {
            return usage(err);
        }
        if (!fileName.endsWith(".mj")) {
            err.println("The source file must have the .mj extension: " +
fileName);
            return EXIT_USAGE;
        }

        String source;
        try {
            source = readSource(Path.of(fileName));
        } catch (IOException e) {
            err.println("Cannot read file '" + fileName + "': " +
e.getMessage());
            return EXIT_USAGE;
        }

        try {
            List<Token> tokens = new Lexer(source).scanTokens();

            if (showTokens) {                       // this option stays for
the whole project
```

```
                for (Token token : tokens) {
                    out.println(token);
                }
                return EXIT_OK;
            }

            Program program = new Parser(tokens).parse();

            if (showAst) {                          // this option stays for
the whole project
                out.println(AstPrinter.print(program));
                return EXIT_OK;
            }

            // SPRINT 3: the SemanticAnalyzer and CodeGenerator are added
here.
            // SPRINT 4: MiniVM is added here (and its VmException is caught
below).
            out.println("Parsing finished: " + program.statements().size()
                    + " statements. (The semantic analyzer is not built
yet.)");
            return EXIT_OK;
        } catch (CompileException e) {
            err.println(e.format());
            return EXIT_ERROR;
        }
    }

    private static String readSource(Path path) throws IOException {
        String text = Files.readString(path, StandardCharsets.UTF_8);
        if (text.startsWith("\uFEFF")) {            // remove the invisible
BOM some Windows editors add
            text = text.substring(1);
        }
        return text;
    }

    private static int usage(PrintStream err) {
        err.println("Usage: java -jar minijava-compiler.jar [--tokens] [--
ast] <file.mj>");
        return EXIT_USAGE;
    }
}
```

Important parts:

- The parser needs just **two lines**: `new Parser(tokens).parse()`. This is the pipeline idea: the output of one stage is the input of the next.
- The same `catch (CompileException e)` now also catches `ParseException`, with no change. This is why we made one base class in Sprint 1.
- `--tokens` and `--ast` stay for the whole project. They are your "X-ray" for each stage.

Create `examples/variables.mj`:

```
// variables.mj
int a = 10;
int b = 20;
a = a + b;
print(a);
print(a * 2 - b);
```

Create `examples/errors/missing_semicolon.mj`:

```
int x = 5
print(x);
```

Create `examples/errors/unmatched_paren.mj`:

```
print((2 + 3);
```

Step 7: Run and verify

1. Run all tests (Maven panel: `Lifecycle > test`, or `mvn test`). Expected:

```
[INFO] Tests run: 50, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

That is 2 in `TokenTest`, 17 in `LexerTest` and 31 in `ParserTest`.

2. See the tree of `hello.mj`. Program arguments: `--ast examples/hello.mj`. Expected:

```
(var age 21)
(var marks 85)
(print age)
(print marks)
(print (+ 10 (* 20 3)))
```

3. Precedence check. Program arguments: `--ast examples/arithmetic.mj`. Expected:

```
(print (+ 2 (* 3 4)))
(print (* (+ 2 3) 4))
(print (- (- 10 4) 3))
(print (+ (neg 5) 8))
```

Read each line and say in your own words which operation happens first. This is the best exercise of this sprint.

4. Assignment. Program arguments: `--ast examples/variables.mj`. Expected:

```
(var a 10)
(var b 20)
(assign a (+ a b))
(print a)
(print (- (* a 2) b))
```

5. Syntax errors (shown on the error stream):

Program arguments	Expected message
`examples/errors/missing_semicolon.mj`	`[Syntax Error] line 1, column 10: Expected ';' after the variable declaration, but found 'print'`
`examples/errors/unmatched_paren.mj`	`[Syntax Error] line 1, column 14: Expected ')' after the value to print, but found ';'`
`examples/errors/bad_char.mj`	`[Lexical Error] line 1, column 11: Unexpected character '@'` (still works from Sprint 1)

6. Normal run. Program arguments: `examples/hello.mj`. Expected:

```
Parsing finished: 5 statements. (The semantic analyzer is not built yet.)
```

7. Optional jar check. `mvn clean package`, then `java -jar target/minijava-compiler.jar --ast examples/variables.mj`. The exit code of an error example should be `1`.

Step 8: Commit, merge and tag

Small commits as you go:

After step	Commit message
1	`feat: add ParseException`
2	`feat: add AST nodes and operators`
3	`feat: add AstPrinter`
4	`feat: add recursive-descent Parser`
5	`test: add Parser tests`
6	`feat: connect Parser in Main and add --ast option`

When all checks pass:

```
git push -u origin feature/sprint-2-parser
```

Open a **Pull Request** into `main` on GitHub, read your own changes once, and merge it. Then:

```
git checkout main
git pull
git tag sprint-2
git push origin sprint-2
```

5. Common Problems

Problem	Fix
"The switch expression does not cover all possible input values"	A kind is missing in `AstPrinter`, or a record is missing from `permits` in `Expr` or `Stmt`
"class is not allowed to extend sealed class"	The record is not in the `permits` list, or it is in a different package. All AST files must be in `parser.ast`
`cannot find symbol: ParseException`	Check the file is in `diagnostics`, and the import in `Parser.java`
A test says `expected: <...> but was: <...>`	Compare the two texts character by character. Common cause: a missing `else if` or a wrong helper in `Parser`. Tell me the test name and the message
Error column is off by one	Check `Token.endColumn()` from Sprint 1 and `errorAfterPrevious` in `Parser`
`Index -1 out of bounds`	`previous()` was called before any token was read. Check that you call `advance()` or `match(...)` before `previous()`
`--ast` prints nothing	The file is empty or has only comments. An empty program prints an empty line

6. Sprint Review Checklist

Before you tell me "Sprint 2 done", confirm:

- [ ] `mvn test` shows 50 passed tests
- [ ] `--ast` output for `hello.mj`, `arithmetic.mj` and `variables.mj` matches this guide
- [ ] The two syntax error messages match this guide
- [ ] I can explain why `expression()` calls `term()`, and why that gives `*` priority over `+`
- [ ] I can explain why a `while` loop gives left-to-right order for `10 - 4 - 3`
- [ ] I can explain why a missing `;` is reported on the earlier line
- [ ] I can explain what a `sealed interface` gives us

- [ ] Code is merged to `main` and tagged `sprint-2`

Next: Sprint 3 builds the semantic analyzer (symbol table and checks), the instruction set and the code generator. They all walk the tree the same way `AstPrinter` does: one `switch` on the kind of node, calling themselves for the inside parts.